Alphafold: Multimodal DTI

Evoformer와 ChemBERTa를 융합한
표적 단백질-화합물 결합 친화도 예측

Multimodal 팀 : 김보형 , 이재권

중간 발표 | 2026년 4월

연구 배경 —신약 개발의 핵심 과제

고비용의 실험 기반 스크리닝 (HTS)

신약 개발 파이프라인에서 특정 약물 후보(화합물)가 표적 단백질에 얼마나 강하게 결합하는지를 예측하는 것은 핵심 병목입니다. 전통적인 실험 기반 스크리닝은 수백만 개의 화합물을 물리적으로 테스트해야 합니다.

기존 딥러닝 DTI 모델의 한계

대부분의 기존 접근법은 단백질과 화합물을 각각 독립적인 1D 시퀀스로만 처리합니다. 이로 인해 실제 결합 메커니즘에서 가장 중요한 잔기-원자 수준의 공간적 상호작용 정보를 포착하지 못하는 한계가 있습니다.
.

본 프로젝트의 제안: Multimodal DTI

단백질의 진화적 정보(MSA)와 화합물의 화학적 구조(SMILES)를 각각 전문화된 인코더로 처리한 뒤, Cross-Attention 기반 융합을 통해 잔기- 원자 수준의 상호작용을 명시적으로 모델링하는 멀티모달 파이프라인을 구축합니다.

10년

평균 신약 개발 기간

26억 달러

평균 개발 비용

수백만 개

물리적 테스트 필요 화합물 수

연구 목표 — 무엇을 예측하는가

Contact Map Prediction

Evoformer가 MSA로부터 추출한 pair representation을 기반으로, 단백질 내 잔내 잔기 쌍(i, j)이 3D 공간에서 8Å 이내에 위치하는지를 이진 분류합니다.
이는 모델이 단백질의 3D 기하학적 구조를 올바르게 학습하고 있는지를 검증하는 보조 과제(auxiliary task)로 기능합니다.

Binding Affinity Prediction

단백질-화합물 쌍에 대해 pIC50 (= -log10(IC50)) 값을 회귀 예측합니다.
이 값이 높을수록 해당 화합물이 표적 단백질에 강하게 결합함을 의미하며, 이는 곧 약물 후보로서의 잠재력을 나타냅니다. 본 프로젝트의 핵심 목표입니다.

예측 목표	출력 형태	손실 함수	생물학적 의미
Contact Map	[B, L, L] 이진 행렬	BCE (pos_weight=5.0)	잔기 간 3D 공간적 근접성
Binding Affinity	[B] 스칼라 (pIC50)	MSE / Huber / Pearson	약물-표적 간의 결합 강도

설계 전략 — AlphaFold에서 무엇을 가져왔는가

AlphaFold2의 Evoformer를 차용하되, Structure Module을 제거하는 경량화 전략

AlphaFold2의 전체 아키텍처는 크게 Evoformer Stack과 Structure Module로 구성됩니다. Structure Module은 3D 좌표를 직접 예측하는 모듈로, TPU 수준의 대규모 연산 자원을 요구합니다.

본 프로젝트의 핵심 설계 결정은 Structure Module을 완전히 제거하고, Evoformer의 표현 학습 능력만을 활용하는 것입니다.

Evoformer가 출력하는 pair representation(z_ij)은 이미 잔기 간 거리 정보를 내재적으로 인코딩하고 있으므로, 명시적인 3D 좌표 예측 없이도 Contact Map을 추출할 수 있습니다.

이렇게 절약된 연산 자원을 화합물 인코딩(ChemBERTa)과 Cross-Attention 투자하여, 단백질-화합물 상호작용 예측이라는 실질적인 목표에 집중합니다.

구성 요소	AlphaFold2 원본	본 프로젝트 (Alphafold-Stark)
Evoformer Blocks	48 blocks	6 blocks (경량화)
Structure Module	포함 (3D 좌표 예측)	제거
화합물 인코더	없음	ChemBERTa 추가
출력	3D 구조	Contact Map + pIC50

전체 아키텍처 개요

3개의 모듈이 순차적으로 연결되는 멀티모달 파이프라인

Protein MSA [B, D, L, 22]

↓

Evoformer Stack (6 blocks)

```
single_repr [B, L, d]
contact_logits [B, L, L]
```

Compound SMILES [B, S]

↓

ChemBERTa Encoder (pre-trained)

```
token_embeddings [B, S, d]
cls_embedding [B, d]
```

Cross-Attention Fusion

```
Protein residues ↔ Compound atoms (bidirectional)
fused [B, 256]
```

Affinity Head (MLP)

```
512 → 256 → 128 → 1
predicted pIC50 [B]
```

모듈 1 — Evoformer Stack 내부 구조

Evoformer Block은 MSA Track과 Pair Track의 교차 업데이트로 구성된다

MSA Track

$m \in \mathbb{R}^{[B \times D \times L \times d_msa]}$

- **Row Attention:** 각 서열 내에서 잔기 위치 간 어텐션을 수행하며, pair representation(z_{ij})을 bias로 주입하여 구조적 정보를 반영합니다.
- **Column Attention:** 각 잔기 위치에서 정렬된 서열들 간 어텐션을 수행하여 진화적 화적 공변이(co-variation)를 포착합니다.

Pair Track

$z \in \mathbb{R}^{[B \times L \times L \times d_pair]}$

- **Outer Product Mean:** MSA 트랙의 정보를 Pair 트랙으로 전달하는 다리 역할니다.
- **Triangle Updates:** Triangle Multiplicative Update와 Triangle Self-Attention을 통해을 통해 3D공간의 기하학적 일관성(삼각 부등식)을 강제합니다.

단일 블록 연산 순서 (OPERATION SEQUENCE)

MSA Track	Row Attention → Column Attention → MSA Transition (FFN)
MSA → Pair	Outer Product Mean
Pair Track	Tri-Mul (outgoing) → Tri-Mul (incoming) → Tri-Attn (row) → Tri-Attn (col) → Pair Transition

핵심 수학 — Scaled Dot-Product Attention

sqrt(d_k) 스케일링 없이는 softmax가 포화되어 그래디언트가 소실된다

$$\text{Attention}(Q, K, V) = \text{softmax}(QK^T / \text{sqrt}(d_k)) \cdot V$$

- 차원 설정: d_k = d_msa / num_heads = 256 / 8 = 32
- 스케일링의 필요성: QK^T의 내적 값은 차원 d_k에 비례하여 증가합니다. sqrt(d_k)로 나누지 않으면 softmax 입력이 극단적으로 커져 출력이 one-hot 벡터에 가까워집니다.
- 그래디언트 소실 방지: softmax가 포화(saturation)되면 역전파 시 그래디언트가 트가 거의 0이 되어 학습이 불가능해집니다. 스케일링은 분산을 1로 유지하여 안정적인 학습을 보장합니다.

- Gating 메커니즘: Evoformer의 모든 어텐션 모듈에는 sigmoid gate가 적용 어텐션 출력의 각 차원을 선택적으로 통과시킵니다. 이는 정보의 흐름을 동적으로 제어하는 역할을 합니다.

```
# models/evoformer.py # Apply element-wise gating to attention output gate =
gate = torch.sigmoid(self.to_gate(m_norm)) # attn_out: [B, L, D, d_msa] attn_out =
attn_out = gate * attn_out # element-wise gating out = self.out_proj(attn_out)
self.out_proj(attn_out)
```

핵심 수학 — Triangle Multiplicative Update

삼각 부등식을 인코딩하여 pair representation의 기하학적 일관성을 보장한다

3D 기하학의 신경망 인코딩

Triangle Multiplicative Update는 AlphaFold2의 가장 독창적인 연산 중 하나로, 3D 공간의 삼각 부등식(triangle inequality)을 pair representation에 직접 인 인코딩합니다.

생물학적 직관

잔기 i가 중간 잔기 k에 가깝고, k가 잔기 j에 가깝다면, i와 j도 가까울 가능성이 높습니다. 이는 3D 공간에서의 삼각 부등식 ||d(i,j)|| ≤ ||d(i,k)|| + 신경망이 학습할 수 있도록 유도합니다.

양방향 기하학적 정보

Outgoing variant(z_ik, z_jk)와 Incoming variant(z_ki, z_kj)를 순차적으로 적용하여, 동일한 논리를 반대 방향에서도 적용함으로써 양방향 기하학적 정보를 모두 반영합니다.

OUTGOING VARIANT 핵심 수식

$$z_{ij} \leftarrow \Sigma_k (gate_{ik} \cdot proj_{ik}) \cdot (gate_{jk} \cdot proj_{jk})^T$$

```
# models/evoformer.py — TriangleMultiplicativeUpdate if self.mode == "outgoing": # z_ij ← sum_k left_ik * right_jk out = torch.einsum("bidc,bjdc->bijc", left, right) else: # incoming # z_ij ← sum_k left_ki * right_kj out = torch.einsum("bdic,bdjc->bijc", left, right)
```

핵심 수학 — Outer Product Mean

MSA 트랙의 진화적 정보를 Pair 트랙으로 전달하는 다리 역할

정보 전달의 핵심 연산

Outer Product Mean은 MSA representation에서 pair representation으로 정보로 정보를 전달하는 핵심 연산입니다.

공진화(Co-evolution) 패턴 포착

이 연산은 MSA의 D개 서열에 걸쳐 잔기 i와 j의 공진화 패턴을 외적(outer product)으로 포착합니다.

물리적 근접성과의 상관관계

진화적으로 연결된 잔기 쌍은 MSA에서 유사한 변이 패턴을 보이며, 이는 3D 공간에서의 물리적 근접성과 높은 상관관계를 가집니다.

OUTER PRODUCT MEAN 수식

z_ij += Linear(mean_over_D(outer(a_i, a_j)))

여기서 a = Linear(m) ∈ ℝ^[B×D×L×c] (c=32, 효율성 확보)

```
# models/evoformer.py — OuterProductMean # einsum으로 효율적 외적 계산 # a_i ⊗ b_j
→ [B, D, L, L, c, c] outer = torch.einsum("bdip,bdjq->bdijpq", a / D, b) # Sum over D
(weighted mean already applied via w) outer = outer.sum(dim=1) # D 차원 평균 [B, L, L, c,
c] # Flatten c×c and project to d_pair outer = outer.reshape(B, L, L, self.c * self.c) return
self.out_proj(outer) # [B, L, L, d_pair]
```

모듈 2 — ChemBERTa 화합물 인코더

사전 학습된 화학 언어 모델로 SMILES 문자열의 분자 구조를 인코딩한다

ChemBERTa 개요

ChemBERTa(Chithrananda et al., 2020)는 PubChem의 7,700만 개 SMILES 문자열에 대해 Masked Language Modeling으로 사전 학습된 RoBERTa 기반 트랜스포머입니다. 이 모델은 화학 구조, 입체화학, 작용기 등의 분자 수준 정보를 토큰 임베딩으로 인코딩하여, 단순한 1D 시퀀스를 넘어선 풍부한 화학적 문맥을 제공합니다.

표현 추출 (Representation Extraction)

- `[CLS]` 토큰 임베딩: 화합물 전체의 글로벌 표현(Global representation)으로 사용됩니다.
- `token_embeddings`: 원자 수준의 로컬 표현(Local representation)으로, 이후 Attention에서 단백질 잔기와의 상호작용을 계산하는 데 사용됩니다.

본 프로젝트의 설계 결정

- 부분 동결 (Partial Freeze): 하위 6개 트랜스포머 레이어를 동결하여 일반적인 화학 지식을 보존합니다. 이는 소규모 DTI 데이터셋에서의 catastrophic forgetting을 방지하기 위한 핵심 전략입니다.
- 384차원 → 256차원 (projection): ChemBERTa의 384차원 출력을 fusion 모듈의 256차원으로 변환합니다.

(LayerNorm → Linear → GELU → Dropout → Linear)

ChemBERTa 설정 요약	
기반 모델	seyonec/ChemBERTa-zinc-base-v1
동결 레이어	하위 6개 (+ embedding layer)
출력 차원	384 → 256 (projection)
최대 SMILES 길이	256 tokens

모듈 3 — Cross-Attention Fusion

단백질 잔기와 화합물 원자 간의 명시적 상호작용 모델링

결합 메커니즘 모사

Cross-Attention Fusion은 단백질(Evoformer)과 화합물(ChemBERTa)의 표현을 결합하는 핵심 모듈입니다. 단순한 벡터 연결(concatenation)을 넘어, 실제 물리적 결합 메커니즘을 모사합니다.

양방향 어텐션 (Bidirectional)

Protein-to-Compound: 단백질의 각 잔기가 화합물의 어느 원자와 상호작용하는지 파악합니다.
Compound-to-Protein: 화합물의 각 원자가 단백질의 어느 잔기와 결합하는지 파악합니다.

최종 융합 (Fusion)

양방향 어텐션의 결과를 결합하고, Global Average Pooling을 통해 최종적으로 256차원의 융합 벡터(fused vector)를 생성합니다.

CROSS-ATTENTION 수식

$$\mathrm{Attn}(Q, K, V) = \mathrm{softmax}(QK^T / \sqrt{d_k})V$$

P2C: Q=Protein, K=Compound, V=Compound
C2P: Q=Compound, K=Protein, V=Protein

```
# models/cross_attention.py — CrossAttentionFusion # Protein-to-Compound Attention p2c_out,
_ = self.p2c_attn( query=prot_repr, key=comp_repr, value=comp_repr,
key_padding_mask=comp_mask ) # Compound-to-Protein Attention c2p_out, _ = self.c2p_attn(
query=comp_repr, key=prot_repr, value=prot_repr, key_padding_mask=prot_mask ) # Global
Average Pooling & Concatenation p2c_pooled = p2c_out.mean(dim=1) c2p_pooled =
c2p_out.mean(dim=1) fused = torch.cat([p2c_pooled, c2p_pooled], dim=-1)
```

Affinity Head — 최종 예측

융합된 벡터로부터 최종 pIC50 스칼라 값을 회귀 예측하는 MLP

점진적 차원 축소 (Dimensionality Reduction)

Cross-Attention Fusion 모듈에서 출력된 512차원의 융합 벡터를 입력으로 받습니다. 3계층 MLP MLP를 통해 512 → 256 → 128 → 1로 차원을 점진적으로 축소하며 비선형 결합 친화도 함수를 를 근사합니다.

정규화 및 활성화 함수 (Regularization & Activation)

각 은닉층 뒤에는 GELU 활성화를 사용하여 부드러운 비선형성을 제공하고, 과적합 방지를 위해 Dropout(p=0.1)을 적용합니다.

최종 출력: pIC50

최종 출력은 단일 스칼라 값인 pIC50 (-log10(IC50))입니다. 값이 클수록 화합물이 표적 단백질에 에 더 적은 농도로 강하게 결합함을 의미합니다.

데이터 파이프라인 — 전처리와 C++ 가속

파이썬의 GIL 한계를 극복하기 위한 C++ 기반 멀티스레드 MSA 파서 도입

CPU 병목 현상 (Bottleneck)

단백질의 다중 서열 정렬(MSA) 데이터인 .a3m 파일은 크기가 수백 MB에 달할 수 있습니다. 파이썬의 내존 파서로 이를 처리할 경우, Global Interpreter Lock Lock (GIL)로 인해 멀티코어 활용이 제한되어 데이터 로딩 속도가 GPU(A100)의 연산 속도를 따라가지 못하는 병목이 발생합니다.

C++ 가속 솔루션

- pybind11 연동: C++로 작성된 고성능 파서를 파이썬 모듈로 컴파일하여 매끄게 통합했습니다.
- 메모리 맵핑 (mmap): 대용량 파일을 메모리에 직접 매핑하여 디스크 I/O 오버헤드를 최소화했습니다.
- OpenMP 병렬화: 여러 서열을 동시에 파싱하고 원-핫 인코딩(One-hot을 수행하여 CPU 코어를 100% 활용합니다.

MSA 파싱 성능 비교 (100MB .a3m 기준)

구현 방식	소요 시간	GPU 대기 시간
Python (BioPython)	~4.5 초	발생 (심각)
C++ (본 프로젝트)	~0.2 초 (22배 향상)	없음 (Zero-wait)

```
# data/dataset.py — C++ 파서 활용 import fast_msa_parser # pybind11로 빌드된 C++ 모듈
def load_msa(self, a3m_path): # C++ 백엔드에서 멀티스레딩으로 파싱 및 텐서 변환 # 반환
값: [D, L, 22] 형태의 numpy 배열 msa_array = fast_msa_parser.parse_a3m( a3m_path,
max_seqs=self.max_msa_seqs, num_threads=8 ) return torch.from_numpy(msa_array)
```

커리큘럼 학습 전략

Cold-start 문제 해결을 위한 3단계 점진적 학습

초기부터 긴 단백질 서열(L>1000)을 학습하면 OOM(Out of Memory)과 극심한 학습 불안정성(Cold-start)이 발생합니다.
이를 해결하기 위해 시퀀스 길이와 크롭 크기를 점진적으로 늘려가는 커리큘럼 학습(Curriculum Learning)을 도입했습니다.

STAGE 1
Warm-up

max_seq_len	256
crop_size	128
batch_size	8

짧은 서열과 큰 배치 사이즈를 사용하여 빠르고 안정적인 초기 가중치 업데이트를 유도합니다.

STAGE 2
Intermediate

max_seq_len	512
crop_size	256
batch_size	4

중간 길이의 단백질 구조를 학습하며, 모델이 더 넓은 범위의 공간적 상호작용을 포착하도록 합니다.

STAGE 3
Fine-tuning

max_seq_len	1024
crop_size	384
batch_size	2

실제 타겟 단백질의 전체 길이에 가까운 서열을 처리하며, 복잡한 상호작용을 정밀하게 조정합니다.

손실 함수 설계

Contact Map 분류와 Affinity 회귀의 동적 가중치 결합

MULTI-TASK LOSS FUNCTION

$$L_total = w_contact \cdot L_contact + w_affinity \cdot L_affinity$$

```
# training/loss.py — AlphaFoldLoss def forward(self, preds, targets, epoch): # 1. Contact Map Loss (BCE with pos_weight) contact_loss = self.bce_loss( preds['contact_logits'], targets['contact_map'] ) # 2. Affinity Loss (MSE or Huber) affinity_loss = self.mse_loss( preds['pic50'], targets['pic50'] ) # 3. Dynamic Weighting based on epoch w_c, w_a = self.get_weights(epoch) total_loss = (w_c * contact_loss) + (w_a * affinity_loss) return total_loss, contact_loss, affinity_loss
```

L_contact: 보조 과제 (Auxiliary Task)

단백질의 3D 구조적 일관성을 학습하기 위한 손실입니다.

- BCEWithLogitsLoss: 이진 분류 문제(8Å 이내 여부)에 사용됩니다.
- 클래스 불균형 해결: 대부분의 잔기 쌍은 멀리 떨어져 있으므로(Negative), pos_weight= 5.0을 부여하여 Positive 샘플(접촉)에 더 큰 페널티를 줍니다.

L_affinity: 주 과제 (Main Task)

최종 목표인 결합 친화도(pIC50)를 예측하기 위한 손실입니다.

- MSELoss / HuberLoss: 연속적인 스칼라 값을 예측하는 회귀 문제에 사용됩니다. 이상 상치(Outlier)가 많은 데이터셋의 경우 Huber Loss가 더 안정적입니다.

동적 가중치 조절 (Dynamic Weighting)

커리큘럼 학습 단계에 따라 가중치를 동적으로 조절합니다. 학습 초기에는 단백질 구조 학습(w_contact)에 집중하고, 후반부로 갈수록 결합 친화도 예측(w_affinity)의 비중을 높여 최종 성능을 극대화합니다.

분산 학습 및 메모리 최적화

대규모 모델 학습을 위한 시스템 최적화 전략

Distributed Data Parallel (DDP)

다중 GPU 환경에서 데이터를 분할하여 병렬로 처리합니다. 각 GPU가 독립적인 배치를 처리하고 역전파 단계에서 그래디언트를 동기화(All-Reduce)하여 대규모 데이터셋의 학습 속도를 극대화합니다.

Automatic Mixed Precision (AMP)

FP16(반정밀도)과 FP32(단정밀도)를 혼합 사용하여 메모리 사용량을 절반으로 줄이고 연산 속도를 향상시킵니다. Loss Scaling 기법을 적용하여 FP16의 정밀도 한계로 인한 Underflow 문제를 방지합니다.

Gradient Checkpointing

Evoformer의 깊은 레이어에서 발생하는 메모리 부족(OOM)을 방지합니다. 순전파 시 중간 활성화 값을 모두 저장하지 않고, 역전파 시 필요한 부분만 재계산하여 메모리 사용량을 O(N)에서 O(sqrt(N))으로로 대폭 감소시킵니다.

Target Hardware: NVIDIA A100 80GB Multi-GPU Cluster

프로젝트 코드 구조

유지보수성과 확장성을 고려한 모듈화된 아키텍처

```
Alphafold-Stark/ ├──configs/ # 하이퍼파라미터 관리 │ └── default.yaml ├── data/ # 데이터 파이프라인 │ ├──dataset.py │ └─fast_msa_parser.cpp ├──models/ # 핵심 신경망 아텍처 │ ├── evoformer.py # 단백질 인코더 │ ├── chemberta.py # 화합물 인코더 │ ├── cross_attention.py # 융합 모듈 │ └── dti_model.py # 전체 모델 통합 ├── training/ # 학습 로직 │ ├──trainer.py # DDP/AMP 학습 루프 │ ├──loss.py # Multi-task 손실 함수 │ └── curriculum.py # 점진적 학습 스케줄러 ├──scripts/ # 실행 스크립트 │ ├── train.py │ └── evaluate.py └── tests/ # 단위 테스트 (pytest) ├──test_evoformer.py └─test_loss.py
```

관심사의 분리 (Separation of Concerns)

- Models: 각 신경망 컴포넌트를 독립적인 파일로 분리하여 단위 테스트와 재사용성을 용이하게 했습니다.
- Training: 복잡한 분산 학습 로직(DDP, AMP)과 커리큘럼 스케줄링을 모델 아키텍처와 완전히 분리했습니다.
- Configs: 하드코딩된 변수를 제거하고 YAML 파일을 통해 모든 하이퍼파라터를 중앙 집중식으로 관리합니다.

코드베이스 통계 (Codebase Statistics)	
총 파일 수 (Python + C++)	**28 개**
총 코드 라인 수 (LOC)	**5,175 줄**
테스트 커버리지 목표	**> 85%**

평가 지표

모델 성능 검증을 위한 5가지 표준 DTI 벤치마크 지표

MSE

Mean Squared Error

예측값과 실제값 차이의 제곱 평균입니다. 큰 오차에 더 큰 페널티를 부여합니다.

Lower is better

RMSE

Root Mean Squared Error

MSE의 제곱근으로, 실제 단위와 같아 오차 규모 파이 용이합니다.

Lower is better

MAE

Mean Absolute Error

절대오차의 평균으로, 이상치에 덜 민감합니다.

Lower is better

Pearson R

Pearson Correlation

예측값과 실제값 간의 선형 상관관계를 -1에서 1 사로 평가합니다.

Higher is better (Max: 1.0)

CI

Concordance Index

예측 순위와 실제 순위의 일치 비율로, 스크리닝 성능을 나타냅니다.

Higher is better (Max: 1.0)

테스트 및 검증

pytest를 활용한 체계적인 단위 테스트 및 수치적 안정성 검증

모듈 정확성 (Module Correctness)

Evoformer, ChemBERTa, Cross-Attention 등 각 신경망 컴포넌트가 독립적으로 올바르게 작동하는지 검증합니다. 특히 입출력 텐서의 형태(Shape)가 설계된 아키텍처와 정확히 일치하는지 확인합니다.

수치적 안정성 (Numerical Stability)

손실 함수(Loss) 계산 시 NaN이나 Inf 값이 발생하지 않는지 검증합니다. 또한, AMP 환경에서 FP16 연산이 언더플로우 없이 안정적으로 수행되는지 테스트합니다.

엣지 케이스 (Edge Cases)

극단적으로 짧거나 긴 시퀀스 길이, 그리고 배치 내에서 길이가 다른 데이터를 처리할 때 패딩 마스크(Padding Mask)가 어텐션 연산에 올바르게 적용되는지 검증합니다.

```
# tests/test_evoformer.py import torch import pytest from models.evoformer import
EvoformerEncoder @pytest.fixture def evoformer_model(): return EvoformerEncoder(
c_m=256, c_z=128, num_blocks=4 ) def test_evoformer_forward_shape(evoformer_model):
batch_size = 2 seq_len = 128 # Dummy inputs msa_repr = torch.randn(batch_size, seq_len, 256)
pair_repr = torch.randn(batch_size, seq_len, seq_len, 128) mask = torch.ones(batch_size,
seq_len) # Forward pass out_msa, out_pair = evoformer_model( msa_repr, pair_repr, mask ) #
Shape assertions assert out_msa.shape == (batch_size, seq_len, 256) assert out_pair.shape ==
(batch_size, seq_len, seq_len, 128) # Numerical stability check assert not
torch.isnan(out_msa).any()
```

향후 계획 및 요약

프로젝트 핵심 성과 및 다음 단계

핵심 성과 요약

멀티모달 아키텍처 구축

단백질(Evoformer)과 화합물(ChemBERTa)의 1D/2D/3D특성을 동시 학습하는 통합 파이프라인 완성

물리적 상호작용 모사

Cross-Attention Fusion을 통해 단백질 잔기와 화합물 원자 간의 양방향 결합 합 메커니즘 구현

학습 안정성 확보

Cold-start 문제 해결을 위한 Curriculum Learning 및 Multi-task Loss(Contact Loss(Contact Map + Affinity) 도입

시스템 최적화

DDP, AMP, Gradient Checkpointing을 적용하여 대규모 분산 학습 환경(Multi-GPU) 구축 완료

향후 계획 (Future Works)

대규모 데이터셋 사전 학습

PDBbind, BindingDB 등 대규모 공개 데이터셋을 활용한 Pre-training 진행 및 행 및 모델 일반화 성능 향상

하이퍼파라미터 최적화

Ray Tune 등을 활용하여 최적의 네트워크 구조(Layer 수, 차원 크기) 및 학습 파라미터 자동 탐색

실험적 검증 (In vitro)

예측 성능이 우수한 후보 물질에 대해 실제 결합 친화도 실험을 수행하고, 결 결과를 모델에 반영하는 피드백 루프 구축

생성 모델로의 확장

결합 친화도 예측을 넘어, 타겟 단백질 구조에 최적화된 신규 화합물을 설계하는 De novo design 연구로 확장

감사합니다